Qwen3 Loop-Trigger Attention: Trigger End vs Trigger Start

2026-04-15

Object

This note compares the two query positions that matter for the current question on the same saved March 22–23 Qwen3-1.7B instruct prompt-profile rows:

- `trigger_end`: the final token of the twentieth repeated 30-gram
- `trigger_start`: the token immediately before that twentieth repeated copy begins

This report uses `trigger_start` only for the token right before the final repeated copy begins. Everything else is held fixed:

- model: `Qwen/Qwen3-1.7B`
- loop definition: `n=30`, `k=20`
- selection rule: recomputed trigger prefix must exactly match saved `first_loop_prefix_length`
- selected rows: `811 / 820` replayable loop rows
- dataset counts: `GPQA 121`, `AIME 30`, `MATH-500 68`, `MMLU-Pro 130`, `LiveCodeBench 462`
- total prefix lengths: median `9846`, `p95 = 22321`, max `28830`

There is still no extra mid-prefix truncation inside any run. Each forward pass sees the full prompt plus the completion prefix through the saved first loop trigger.

Reconstruction Boundary

The March prompt-profile archives preserve exact `prompt_token_ids`, but they do not preserve exact `completion_token_ids`. For those older rows, the replay path still retokenizes saved `completion_text`.

Dataset	Rollouts	Loop rows	Exact length	+1 hidden stop	Trigger-prefix match
GPQA	792	124	46 / 792	749 / 792	121 / 124
AIME	240	31	38 / 240	234 / 240	30 / 31
MATH-500	2000	68	38 / 2000	1991 / 2000	68 / 68
MMLU-Pro	3200	130	34 / 3200	3191 / 3200	130 / 130
LiveCodeBench	3200	467	356 / 3200	3168 / 3200	462 / 467
Overall	9432	820	512 / 9432	9333 / 9432	811 / 820

So the replay boundary is still: exact prompt IDs, approximate completion IDs on these old archives, and exact trigger-prefix recovery on `811 / 820` loop rows.

Measurement Definition

For each selected row:

1. Recompute the loop trigger with `find_ngram_loop_trigger(..., n=30, k=20)`.
2. Keep only rows whose recomputed `first_loop_prefix` matches the saved `first_loop_prefix_length`.
3. Replay the model on `prompt_token_ids + completion_token_ids[:first_loop_prefix_length]`.
4. Apply an explicit causal mask inside the probe wrapper so keys with `key_position > query_position` are forced to zero.

The two reported query positions are:

- `trigger_end = prompt_len + trigger_end`
- `trigger_start = prompt_len + trigger_start - 1`

Key positions are partitioned into disjoint bins:

- `prompt`
- `previous_loop`
- `current_trigger`
- `other_completion`

`recent_nonloop` is still tracked as a diagnostic subset: the previous 256 completion positions outside the loop bins. But on the collaborator-facing surface here, `other_completion` includes that recent window, so the plotted lines and the main tables stay on a full prefix decomposition.

`last_previous_loop` is still tracked as a subset of `previous_loop`, but it is not plotted as a separate mass line here.

There is no separate self bin. The query token contributes to the region it belongs to. At `trigger_start`, `current_trigger` is exactly zero because the full trigger span is in the future.

Summary statistics are:

- `mean_*_mass`: summed attention mass on that bin, averaged across heads, then across selected rows
- `top1_fraction_*`: fraction of heads whose argmax lands in that bin

Layer-by-Layer Progression

`attention_mass_by_layer_comparison.pdf` plots row-weighted overall bin mass from layer 0 to layer 27 for the two reported query positions. The plotted lines are:

- `trigger_end`: `prompt`, `previous_loop`, `current_trigger`, and `other_completion`
- `trigger_start`: `prompt`, `previous_loop`, and `other_completion`

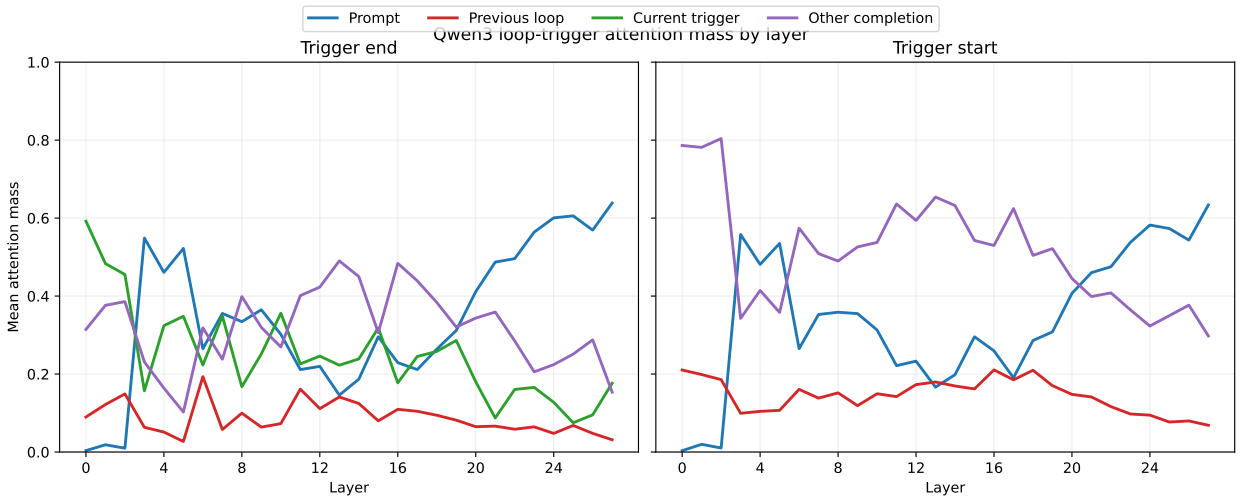

Figure 1: Row-weighted overall attention-mass progression from layer 0 to layer 27 for the two reported query positions. `Other_completion` is the full residual completion mass outside the loop bins, including the recent non-loop window.

Main read from the curves:

- `trigger_end`: previous-loop mass rises early, peaks at layer 6 (`0.193`), and then falls as the final layer becomes prompt-dominant; by layer 27, `current_trigger` (`0.176`) is still larger than `other_completion` (`0.154`).
- `trigger_start`: `current_trigger` is identically `0`, previous-loop mass peaks later at layer 16 (`0.211`), and the strongest competition there is between `prompt` (`0.260`) and `other_completion` (`0.530`).

So the two views differ materially. Moving the query one token earlier removes `current_trigger`, raises final-layer previous-loop mass, and still leaves most of the non-prompt mass in the residual `other_completion` bin.

Final-Layer Overall Comparison

The final-layer summary is layer 27 for both reported objects.

Query	Prev-loop	Prompt	Current-trigger	Other-completion	Top-1 prev	Top-1 prompt	Top-1 current	Top-1 other
`trigger_end`	0.031	0.639	0.176	0.154	0.0003	0.874	0.114	0.011
`trigger_start`	0.069	0.634	0.000	0.298	0.027	0.872	0.000	0.100

This comparison says two separate things:

- moving from `trigger_end` to `trigger_start` removes `current_trigger` entirely, raises final-layer previous-loop mass from `0.031` to `0.069`, and raises residual completion mass from `0.154` to `0.298`;
- prompt tokens remain the dominant late-layer target in both views.

Length-Adjusted Final-Layer Read

Raw mass alone is not enough for the collaborator-facing conclusion, because the four bins have very different lengths on the accessible prefix.

For each query definition and bin, let `token_share` be the row-weighted mean fraction of accessible positions that lie in that bin, and define `enrichment = mean_mass / token_share`. Then `enrichment = 1` is the length-matched uniform-attention baseline. Values above `1` mean the region gets more mass than its size alone would predict; values below `1` mean it gets less.

Query	Prompt share	Prev-loop share	Current-trigger share	Other-completion share
`trigger_end`	0.051	0.069	0.004	0.876
`trigger_start`	0.051	0.069	0.000	0.879

Query	Prompt enrich.	Prev-loop enrich.	Current-trigger enrich.	Other-completion enrich.
`trigger_end`	12.57x	0.46x	42.62x	0.18x
`trigger_start`	12.41x	0.99x	0.00x	0.34x

This changes the interpretation in an important way:

- `other_completion` has large raw mass mainly because it covers about `88-91%` of the visible prefix; after length adjustment it is under-attended in both views.
- `prompt` is the only region that stays strongly over-attended in the final layer in both views.
- `previous_loop` is below the uniform baseline at `trigger_end`, and only about at the baseline overall at `trigger_start`.

Because the bin boundaries are fixed once the query position is fixed, the token shares do not vary by layer. So the length-adjusted curves have the same peak layers as the mass curves; only the vertical scale changes. On that size-adjusted read:

- `trigger_end` previous-loop enrichment still peaks at layer 6, now at `2.80x`;
- `trigger_start` previous-loop enrichment still peaks at layer 16, now at `3.03x`.

Trigger-Start By Dataset

For the `trigger_start` object, `current_trigger` is zero on every dataset by construction, so the dataset-level table only reports the remaining bins.

Dataset	Rows	Prev-loop	Prompt	Other-completion	Top-1 prev	Top-1 prompt	Top-1 other
GPQA	121	0.045	0.648	0.307	0.011	0.872	0.117
AIME	30	0.107	0.628	0.266	0.050	0.869	0.081
MATH-500	68	0.082	0.619	0.299	0.035	0.837	0.128
MMLU-Pro	130	0.032	0.643	0.326	0.005	0.909	0.086
LiveCodeBench	462	0.081	0.630	0.289	0.035	0.868	0.097
Overall row-weighted	811	0.069	0.634	0.298	0.027	0.872	0.100

Dataset-level scientific read:

- `AIME` shows the strongest final-layer previous-loop mass (`0.107`).

- MATH-500 and LiveCodeBench are next strongest at about 0.08.
- MMLU-Pro is the weakest on this object (0.032 prev-loop mass, 0.005 top-1 prev-loop).
- Every dataset still stays prompt-dominant in the final layer: prompt mass is 0.619-0.648, while other_completion ranges from 0.266 to 0.326.

The size-adjusted trigger_start previous-loop read is heterogeneous by dataset:

- AIME: 2.16x
- LiveCodeBench: 1.64x
- MATH-500: 1.12x
- GPQA: 0.39x
- MMLU-Pro: 0.31x

So the overall trigger_start previous-loop enrichment near 1.0x is hiding real spread across datasets rather than proving a uniform mechanism.

Conclusion

The raw-mass read is still true but incomplete:

- the pre-trigger object does show nontrivial attention to previous loops: final-layer previous-loop mass 0.069, final-layer top-1 previous-loop 0.027, and overall previous-loop peak at layer 16 with mass 0.211;
- but the final layer is still mostly prompt-focused rather than previous-loop-focused: final-layer prompt mass 0.634, final-layer residual completion mass 0.298, and final-layer top-1 prompt 0.872.

The length-adjusted read is sharper:

- prompt is still the truly concentrated final-layer target in both views (12.57x at trigger_end, 12.41x at trigger_start);
- other_completion is not a preferred target per token; its large raw mass mostly comes from covering almost the whole prefix;
- previous_loop is not dominant in the final layer overall: it is below baseline at trigger_end and only about baseline overall at trigger_start, even though its mid-stack signal becomes clearly super-baseline and some datasets exceed baseline at trigger_start.

So the strongest honest statement is not "Qwen3 is paying mostly to prompt + other completions rather than previous loops," because that bundles together one strongly over-attended region (prompt) with one strongly under-attended region (other_completion). The stronger-supported statement is:

> At the token immediately before the twentieth repeated copy begins, the final layer is still mostly prompt-focused, while previous-loop evidence is present and peaks in the middle of the stack.

Artifact Bundle

Updated raw/report artifacts now live under:

- `outputs/qwen3_loop_trigger_attention_full_20260414_rerun/attention_length_adjusted_overall`
- `outputs/qwen3_loop_trigger_attention_full_20260414_rerun/attention_length_adjusted_by_data`
- `outputs/qwen3_loop_trigger_attention_full_20260414_rerun/attention_length_adjusted_by_laye`
- `outputs/qwen3_loop_trigger_attention_pre_trigger_start_20260414/` (`trigger_start` bundle)